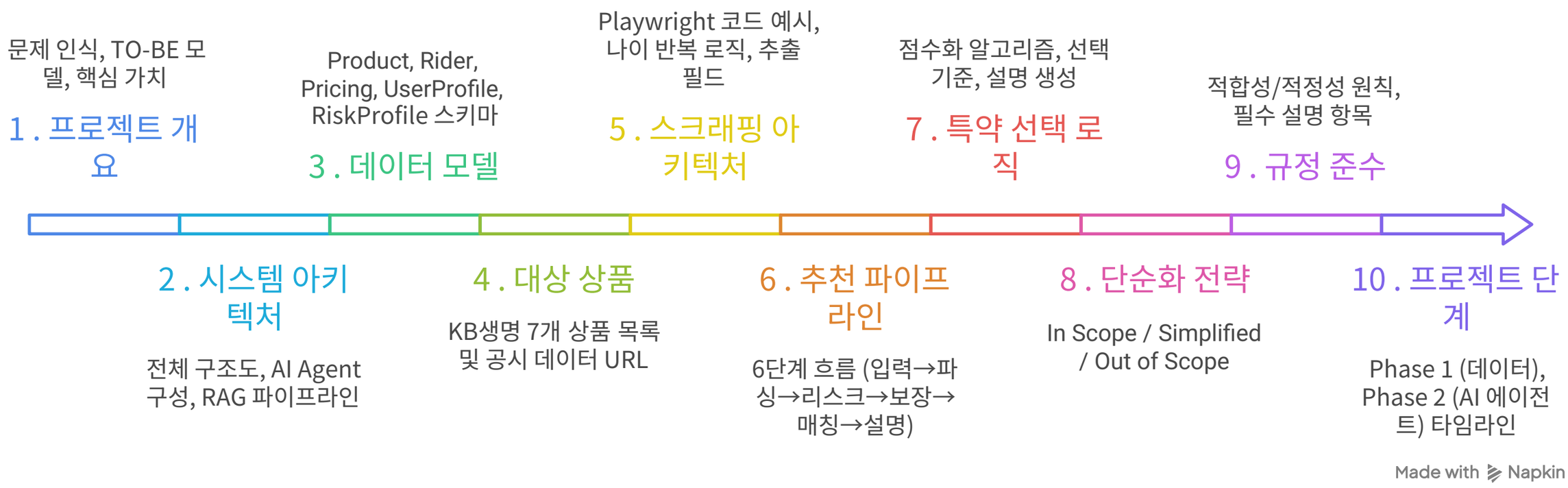
산학 스터디 -트랙 B



핵심 요약

프로젝트 목표: AI가 보험설계사(GA) 역할을 대체하여 고객이 직접 보험을 설계할 수 있는 시스템 구축

Track B 역할:

- 1 . KB생명 7개 상품 웹 스크래핑
- 2 . Playwright로 보험료 자동 수집 (20~64세)
- 3 . PDF 파싱 (약관/요약서)
- 4 . 데이터 구조화 + 벡터 임베딩

기술 스택 추천:

- Scraping: Playwright
- DB: PostgreSQL + Chroma (벡터)
- LLM: GPT-4 / Claude
- Frontend: Next . js

문제 인식

한국 보험 시장에서 GA(법인보험대리점)는 보험사와 고객 사이의 핵심 유통 채널입니다 . 그러나 이 구조에는 근본적인 문제가 존재합 니다 .

- 프로젝트 목표

Track B: 보험 가입 여정 재설계

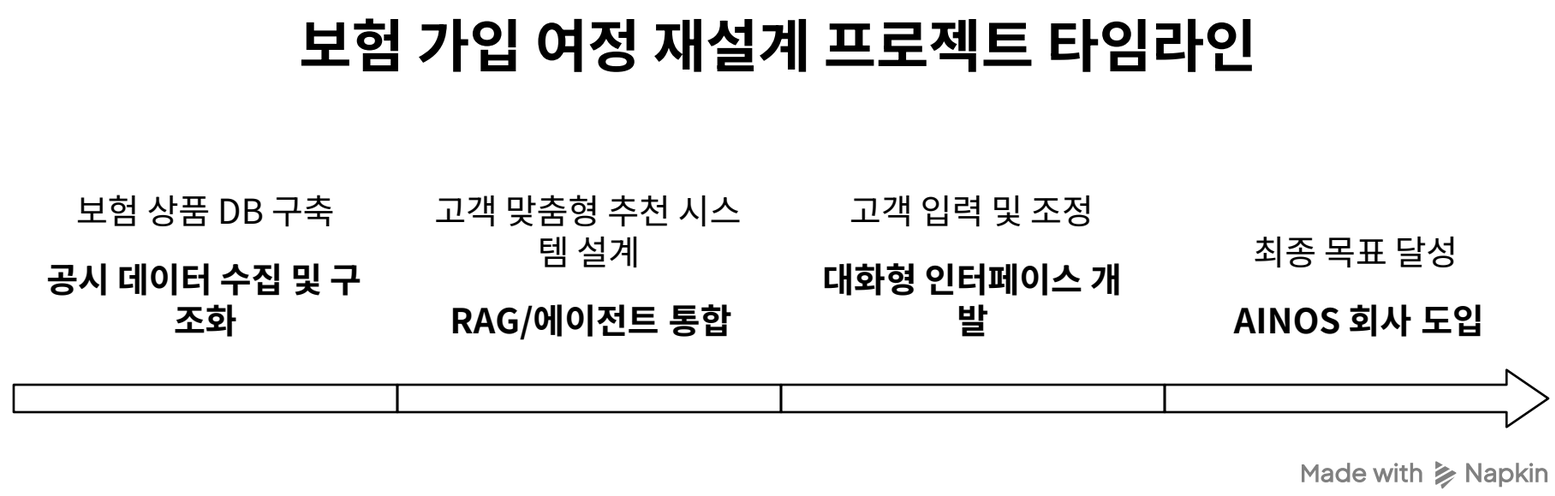
수행 목표:
공시 데이터를 직접 수집·구조화해서 DB를 만들고

,
(스크래핑/파싱/정규화하여 보험 상품 DB를 구축하고)

- 그 DB와 RAG/에이전트를 이용해
- 고객 맞춤형 보험 추천 + 보험료 계산 + 대화형 설계 조정까지 가는 것

데이터 파이프라인 + 추천 로직 + 대화형 인터페이스를 함께 만드는 프로젝트 (고객 입력 → 상품 탐색 → 특약 조합 → 보험료 설명/조정이 가능한 대화형 추천 시스템을 설계)

- → (최종 목표): AINOS회사 측에서 이 인터페이스를 실제로 도입할 수 있도록 하는 것
- 도메인 지식 (기술 + 산업)



1. 기술 용어 기초 지식

구분	웹 스크래핑 (Web Scraping)	데이터 파싱 (Data Parsing)
핵심 동작	웹 사이트에 접속해서 HTML 소스 코드를 다운로드함.	다운로드한 HTML(문자열)에서 원하는 데이터만 추출함.
도구 예시	Playwright, Selenium, Requests	BeautifulSoup, lxml, JSON.parse()
결과물	거대한 HTML 덩어리 (Raw Data)	상품명: "암보험", 가격: 50,000 (Structured Data)

파싱: 비정형 데이터를 스키마에 맞는 형태로 변환한다. 파싱 과정 Ex.

- 1. Raw Data (스크래핑 결과): 50,000원
- 2. Parsing (파싱 과정): 태그 안에 있는 텍스트만 골라내고, '원' 이라는 글자와 쉼표(,)를 제거함.
- 3. Refined Data (가공 결과): price: 50000 (숫자형 데이터로 변환)
- 공시 데이터

Primarykblife.co.kr ↗

KB라이프 공시실

- 과제 대상 7개 상품 상세페이지
- 약관/요약서/설명서 PDF 다운로드
- 보험가격공시 (동적 보험료 계산)
- 상품 보장내용 및 특약 정보

Secondarypub.insure.or.kr ↗

생명보험협회 공시실

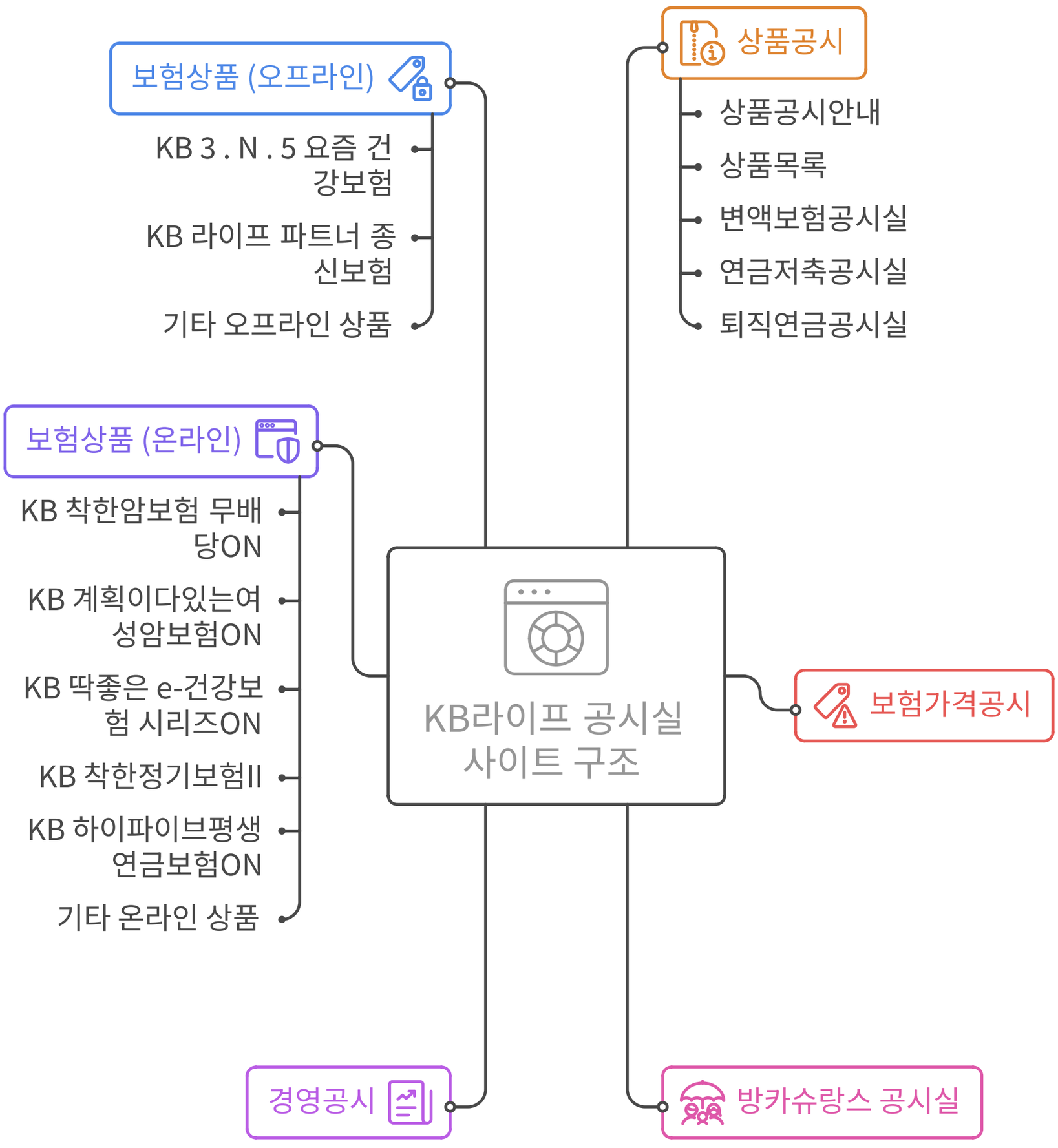
- 암보험 상품비교공시
- 보장성 상품 비교
- 불완전판매비율 비교
- 경영공시 (정기/수시)

Referencefinlife.fss.or.kr ↗

금감원 금융상품한눈에

- 보험상품 통합 검색
- 보험사 간 상품 비교
- 금리/수익률 공시
- 소비자 보호 정보

KB라이프 공시실 사이트 구조
kblife . co . kr



← 추천/계산

```
[웹사이트]↓ (Scraping: Playwright) [Raw Data] ↓ (Parsing & Mapping) [Structured DB] ↓ [Recommendation Logic] ↓ [LLM + RAG] ↑
```

[Vector DB] ← 설명/근거

- 0 . 사전정의
- Product taxonomy 정의
- 데이터 스키마 정의 (Product / Rider / Price)
- MVP 범위 확정 (암보험 1~3개 + 특약 5~10개)
- 1 . 스크래핑 단계

1 -1 . Playwright로 보험료 데이터 수집

[상품 1개 선택]

- → for age in 20~64:
- → 성별 선택
- → 조건 선택 (갱신/비갱신 등)
- → 조회 버튼 클릭

결과 DOM에서 보험료 텍스트 추출

> covered _ event: 암 진단 ,

- → 1 -2 . 분기 IF 데이터가 " 동적 UI 기반 (입력 → 결과) " → Playwright 사용 ELSE IF 데이터가 " 정적 HTML " → requests + bs4 ELSE IF 데이터가 " PDF " → PDF 다운로드 → PDF 파싱 2 -1 . 스키마 매핑 raw 값 → 정의한 schema에 넣기 예: { product _ name: " KB 암보험 " , age: 30 , gender: " female " , premium: 52300 } 3 -1 . Structured DB구축 Product product _ name product _ type Price Table age gender premium Rider rider _ name coverage _ category payout _ condition Parsing 결과 → JSON/CSV → DB insert 4 . 특약 보장 구조화 4 -1 . 데이터 유형별 처리? IF HTML 구조 → BeautifulSoup parsing IF PDF 문서 → PDF parsing (텍스트 추출) 4-2 . 비정형 텍스트 → 정형 데이터로 변환 예: " 암 진단 시 5000만원 지급 " → { rider _ name: " 암 진단 특약 " , "

> payout _ amount: 50000000 } 5 . Rule+ DB기반 추천 로직 [User Input] ↓ 리스크 프로필 생성 ↓ 필요 보장 카테고리 도출 ↓ 상품 후보 필터링 ↓ 특약 조합 선택 ↓ 보험료 계산 6 . RAG 파이프라인 (설명용)

6 -1 . Vector DB 구축

- 공시 PDF 약관 텍스트 상품 설명 → chunking → embedding → vector DB 저장

6 -2 . 역할

> 유저 질문: "

> 왜 이 보험이 좋아요?" → Vector DB에서 관련 문장 검색 → LLM이 근거 기반 설명 생성

- 참고자료 및 기타
- < 내가 생각하는 병목>
- -KB라이프 여러 보험 상품들 종류별로 구분 (구분 방식?에 대한 정의)
- -각 상품별 특약에 대해 데이터 파싱할 구조?? 짜기? (표 , 다이어그램 등 다양한 방식으로 표현되어 있는 특약 속 다양한 데이터를 어떤 형 태로 파싱할지에 대한 고민?) → 이걸 뭘 기준으로 판단해야 될지

(예를 들어 유저 입장에서의 편의+니즈를 반영한 데이터 구조로 파싱할지 아니면 그냥 정형화된 구조를 만들어서 파싱할지?) (데이터 스키마에 대한 고민)

- -RAG파이프라인을 짤 때 knowledge base로 KB라이프 공시 데이터만 사용할건지 . ? 아니면 kb라이프/생명보험협회 공시실/ 금감원 금 융상품 이렇게 셋 다 RAG에 반영할건지
- -이걸 playwright쓰면 스크래핑이 다 되기는하나? (어떤 부분에만 쓸지에 대한 고민? 전체를 다 쓰기에는 너무 오류도 많고 시간이 오래 걸릴거같은데)
- sql url 트랙A

CONTROL _ PLANE _ DATABASE _ URL=postgresql://neondb _ owner:npg _ G0WHV2zkUbsP@ep -proud -king -a1w7a2l4pooler . ap -southeast -1 . aws . neon . tech/neondb?sslmode=require

BRANCH

_

SECRET_
_
KEY=dc18e7750e3b5e4e6ca36ac381543aefb434341570b0cb22a337d48fcbbce209
AUTH _ SECRET=4Ok5 _ RxrcfEbKo17tm7hpZyf2R7NhDYitS0TIDWmGoU
NEON _ API _ KEY=placeholder
NEON _ WEBHOOK _ SECRET=placeholder
CRON _ SECRET=bNveUUWfcToTW6d4f4 _ EV4A1cAhX0Xv86t3hblUrcoA AUTH _ URL=
http://localhost:3000

💢 예상 병목

- . env local만들기

변수	의미
CONTROL _ PLANE _ DATABASE _ URL	Postgres DB 접속 주소
AUTH _ SECRET	인증/세션 서명용 비밀키
BRANCH _ SECRET _ KEY , CRON _ SECRET	내부 보호용 키
AUTH _ URL	로컬 서버 주소

1 . 코드베이스

- GitHub repo / zip 파일 / 프로젝트 폴더

2 . 실행 방법

- npm install
- npm run dev
- 또는 pnpm dev , yarn dev
- 백엔드/프론트 분리 여부

3 . DB 안의 테이블 구조

- 어떤 테이블이 있는지
- 어떤 데이터를 보는지
- read -only인지 write도 되는지

즉 , 지금 받은 건 열쇠고 ,

아직 문(코드) 이랑 방 구조(DB schema) 를 확인해야 해요 .

가장 흔한 실행 흐름

보통은 이렇게 해요 .

1) . env . local 또는 . env 파일 생성

프로젝트 루트에 환경변수 파일을 만들고 넣음 .
예:
.env.local

2) 의존성 설치

npm install
또는
pnpm install

3) 개발 서버 실행

4) 로컬 접속

브라우저에서:

http://localhost:3000

DB가 진짜 연결되는지 확인하는 법

보통 방법은 2개예요 .

방법 A . 앱 실행 후 화면 확인

- 로그인 되는지
- 데이터 리스트가 뜨는지
- 추천 결과가 나오는지

방법 B . DB 직접 접속

Postgres 클라이언트로 연결해서 테이블 확인
툴 예시:

- DBeaver
- TablePlus
- pgAdmin

연결에 필요한 건 원래 DATABASE _ URL 하나면 충분해요 .
연결 후 확인할 것:

- 테이블 목록
- 샘플 row
- 컬럼명
- 보험 상품/가격/특약 관련 데이터 존재 여부

네가 " 이 데이터로 돌려본다 " 는 게 정확히 뭘 의미하냐면

보통 아래 셋 중 하나예요 .

1 . 앱 실행

회사에서 만든 서비스/데모를 로컬에서 띄워보기

2 . DB 탐색

실제 어떤 상품/가격/특약 데이터가 들어있는지 보기

3 . 추천 로직 테스트

입력값을 넣었을 때 어떤 결과가 나오는지 확인

그런데 지금 제일 중요한 문제

너한테 필요한 건 아마 ** " 이걸 어디에 넣고 어떻게 확인하냐 "** 일 텐데 ,
그건 프로젝트 코드 구조를 봐야 정확히 말할 수 있어요 .

예를 들어:

- Next . js 프로젝트인지
- Python 백엔드인지
- Prisma 쓰는지
- Supabase/Neon/Postgres 직접 연결인지

에 따라 방식이 달라져요 .

- 프로젝트 폴더 구조 캡처
- package . json
- README . md
- . env . example 내용
- DB 연결 코드 파일 (lib/db . ts , prisma/schema . prisma , db . py 등)

이걸 돌리려면 추가로 필요해요:

- 프로젝트 코드
- 실행 방법
- DB 구조 확인
- 😵 개발현황

🌱 학회원 전달 Github + 개발 방향성 제시

- 3/22회의

- -GA에 대한 문제→ 명시적으로 말해주지 않는다 (3개 외의 문제는 너네가 찾아라) (문제 정의→ 해결 구조)
- -현재 보험의 유통구조에 대해서 얘기할 생각 (한국 현 상황에 대해 말해줘)→ 짤막한 장표
- -학교 선배님 → 큐앤에이 시간 (학회원들한테 질문 받는 → 팀별로 시간을 주고 나서 질문 받기) 30분 강지민(회사소개+산학소개) / 1시간 김영운 / 30분 팀별 논의/ 30분 결과물 공유
- -오류 미리 병목 예상해서 전달
- -기술적으로 각 프로세스에 대한 설명?? (왜 플레이라이트를 써야하나 -기본적인 설명) (데이터 파싱 전에 스크래핑의 중요성?)